

CS3012E Artificial Intelligence

Imbalanced Disaster Tweet Classification

Submitted by:

B230691CS	Email: poosa_b230691cs@nitc.ac.in
B230487CS	Email: pedakota_b230487cs@nitc.ac.in
B230587CS	Email: syed_b230587cs@nitc.ac.in
B230555CS	Email: shaik_b230555cs@nitc.ac.in
B230567CS	Email: simhadri_b230567cs@nitc.ac.in

Date: April 6, 2026

Contents

1	Introduction	3
2	Dataset Description	3
3	Preprocessing	4
3.1	Data Preprocessing	4
3.2	Vectorization	4
3.3	Train-Test Split	5
4	Methodology	5
4.1	Models Without Imbalance Handling	5
4.1.1	Logistic Regression	5
4.1.2	Random Forest	5
4.1.3	Neural Network (Fixed Architecture)	6
4.2	Models With Imbalance Handling	6
4.2.1	Class Weighting	6
4.2.2	Random Oversampling	6
4.2.3	SMOTE	6
4.3	Model Evaluation	6
5	Experimental Setup	7
5.1	Hardware Requirements	7
5.2	Software Requirements	7
5.3	Model Parameters	7
5.4	Step-wise Execution	8
6	Results	8
6.1	Best Values Observed Across All Models	8
6.2	Part A – Without Imbalance Handling	9
6.3	Part B – With Imbalance Handling	9
6.3.1	Class Weighting	9
6.3.2	Random Oversampling (ROS)	10
6.3.3	SMOTE (Synthetic Minority Oversampling Technique)	10
7	Visualization of Results	11
7.1	Overall Model Comparison	12
7.2	Model Performance Heatmap	13
7.3	Summary of Visualization Observations	14
8	Analysis and Discussion	14

9 Conclusion	15
9.1 Practical Recommendations	15

1 Introduction

The task is to classify tweets as disaster-related or non-disaster-related in a highly imbalanced dataset. The objective is to develop models that can effectively handle this imbalance and improve the detection of disaster-related tweets. For this purpose, Logistic Regression, Random Forest, and a Neural Network with a fixed architecture are used, and their performance is compared along with different imbalance handling techniques.

Objectives:

- Implement Logistic Regression, Random Forest, and Neural Network models.
- Convert text data into numerical form using TF-IDF vectorization.
- Evaluate models using metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC.
- Apply imbalance handling techniques like class weighting, oversampling, undersampling, and SMOTE.
- Compare model performance before and after applying imbalance handling methods.

2 Dataset Description

The dataset used in this project is the Disaster Tweets dataset from Kaggle (tweets.csv), which contains labeled tweets indicating whether they are related to real disasters.

Dataset Summary:

- Total samples: 11,370
- Attributes: *id*, *keyword*, *location*, *text*, *target*
- Missing values: 3,418 in location field
- Class 0 (non-disaster): 9,256 (81.41%)
- Class 1 (disaster): 2,114 (18.59%)

The dataset is highly imbalanced, with a majority of non-disaster tweets. This may lead to high accuracy but poor detection of disaster-related tweets.

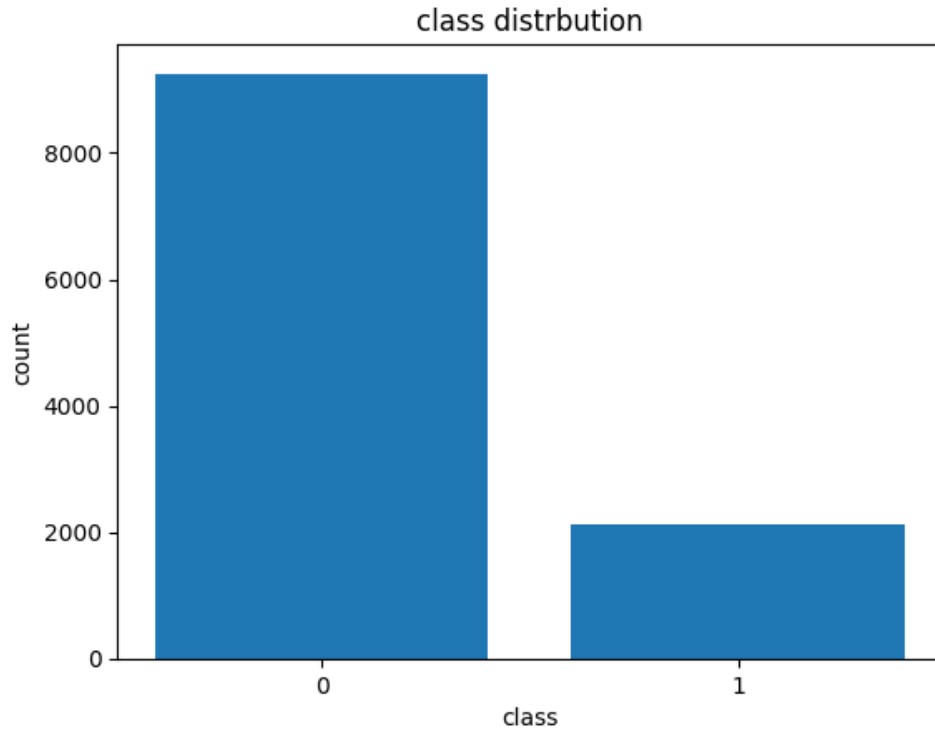


Figure 1: Class Distribution of Dataset

3 Preprocessing

3.1 Data Preprocessing

Data preprocessing involves cleaning and preparing raw text data to make it suitable for analysis and modeling.

- Removal of unwanted elements such as URLs and special characters
- Conversion of text to lowercase for consistency
- Tokenization of text into individual words
- Removal of stopwords (e.g., "the", "is") to reduce noise
- Application of stemming or lemmatization to obtain root words

3.2 Vectorization

Vectorization converts cleaned text into numerical form so that machine learning models can process it.

- Transformation of text into numerical feature vectors
- Use of TF-IDF (Term Frequency-Inverse Document Frequency)
- Enables models to understand and process textual data

3.3 Train-Test Split

The dataset is divided into training and testing sets to evaluate model performance effectively.

- Splitting dataset into training and testing subsets
- Common ratio: 80% training and 20% testing
- Training data used to build the model
- Testing data used to evaluate performance on unseen data
- Helps in avoiding overfitting and improving generalization

4 Methodology

The methodology of this project is divided into two main parts: models trained without imbalance handling and models trained with imbalance handling techniques. The workflow includes preprocessing, vectorization, train-test split, model training, and evaluation.

4.1 Models Without Imbalance Handling

In this stage, models are trained directly on the original dataset without applying any imbalance handling techniques.

4.1.1 Logistic Regression

Logistic Regression is used as a baseline model for binary classification.

- Uses TF-IDF features as input
- The model learns the relationship between input features (TF-IDF vectors) and target labels (0 or 1).
- Trained on the training dataset
- Predictions are made on the test set and evaluated using metrics like accuracy, precision, recall, and F1-score.

4.1.2 Random Forest

Random Forest is an ensemble model that combines multiple decision trees.

- Trains multiple decision trees on random subsets
- Each tree is trained on different subsets of data, and their outputs are combined for the final prediction
- This improves performance and reduces overfitting compared to a single model.
- Evaluated on the test dataset

4.1.3 Neural Network (Fixed Architecture)

A neural network with a predefined structure is used for classification.

- Input layer takes TF-IDF feature vectors
- Hidden layers apply activation functions like ReLU to learn complex patterns.
- The output layer uses a sigmoid function for binary classification.
- Trained and evaluated on training and test data

4.2 Models With Imbalance Handling

To improve performance on the minority class, imbalance handling techniques are applied.

4.2.1 Class Weighting

This method assigns higher importance to minority class samples.

- Higher penalty for misclassifying the minority class
- Helps in learning balanced decision boundaries

4.2.2 Random Oversampling

This technique increases minority class samples by duplication.

- Duplicates existing minority samples
- Balances class distribution
- Improves learning of minority patterns

4.2.3 SMOTE

SMOTE generates synthetic samples for the minority class.

- Creates new samples using interpolation
- Applied after TF-IDF vectorization
- Improves generalization and reduces overfitting

4.3 Model Evaluation

All models are evaluated to compare the performance and effectiveness of imbalance handling.

- Metrics: accuracy, precision, recall, F1-score
- Comparison between models with and without imbalance handling
- Analysis of performance improvements

5 Experimental Setup

This section describes the hardware and software environment used for the implementation of the project, along with the model parameters and step-wise execution process.

5.1 Hardware Requirements

The experiments were conducted on a system with the following specifications:

- Processor: Intel Core i5/i7 or equivalent
- RAM: 8 GB or higher
- Storage: Minimum 10 GB free space
- GPU: Optional (NVIDIA GPU recommended for faster training)

5.2 Software Requirements

The implementation was carried out using the following software tools and libraries:

- Programming Language: Python 3.14.3
- IDE/Editor: Visual Studio Code / Jupyter Notebook
- Libraries:
 - pandas, numpy (data handling)
 - matplotlib, seaborn (visualization)
 - scikit-learn (machine learning models)
 - torch (neural network implementation)
 - nltk (natural language tool kit for text preprocessing)
 - imbalanced-learn (SMOTE and oversampling)

5.3 Model Parameters

The following parameters were used during model training and tuning:

- Vectorization: TF-IDF (max_features = 10000)
- Train-Test Split: 80% training, 20% testing
- Learning Rate: [0.01, 0.001]
- Batch Size: [32, 64]

- Number of Epochs: 10–20
- Optimizer: Adam (Adaptive Moment Estimation), SGD (Stochastic Gradient Descent)
- Activation Function: ReLU (hidden layers), Sigmoid (output layer)
- Regularization: Dropout and L2 regularization

5.4 Step-wise Execution

The complete experiment was executed in the following steps:

1. Load the dataset (tweets.csv) using pandas.
2. Perform data preprocessing (cleaning, tokenization, stopwords removal, stemming).
3. Convert text data into numerical form using TF-IDF vectorization.
4. Split the dataset into training and testing sets.
5. Hyperparameter tuning for LR, RF, NN
6. Train the models (LR, RF, NN) to run without imbalance handling.
7. Apply imbalance handling techniques (class weighting, random oversampling, SMOTE).
8. Retrain the models with balanced data.
9. Evaluate model performance using accuracy, precision, recall, and F1-score.

6 Results

The results of the experiments are obtained from the generated file: **outputs/results/model_comparison** which contains evaluation metrics for all models under different configurations.

6.1 Best Values Observed Across All Models

- Best Accuracy: LR – SMOTE = 0.8821
- Best Balanced Accuracy: RF – ROS = 0.8068
- Best F1 (Macro): LR – SMOTE = 0.7931
- Best F1 (Weighted): LR – SMOTE = 0.8783
- Best ROC-AUC: RF – ROS = 0.9040

- Best PR-AUC: LR – Baseline = 0.7286
- Best Recall (Disaster): RF – ROS = 0.7281
- Best Precision (Disaster): RF – Baseline = 0.7629

6.2 Part A – Without Imbalance Handling

Table 1: Model Comparison (Without Imbalance Handling)

Model	Accuracy	Balanced Acc.	Recall (Disaster)	Recall (Non-Dis.)	F1 Macro	ROC-AUC
Logistic Regression	0.8830	0.7868	0.6336	0.9400	0.7987	0.9011
Random Forest	0.8676	0.6944	0.4184	0.9703	0.7316	0.8985
Neural Network	0.8509	0.7479	0.5839	0.9119	0.7509	0.8616

Observation:

- Logistic Regression achieves the best overall performance in terms of accuracy (0.8830), F1 Macro (0.7987), and ROC-AUC (0.9011).
- Neural Network achieves a higher balanced accuracy (0.7479) and recall (0.5839) than Random Forest, indicating better minority class sensitivity.
- Random Forest has the lowest balanced accuracy (0.6944) and recall (0.4184), showing strong bias toward the majority class.
- All models are biased towards the majority class without imbalance handling, as reflected by relatively low disaster recall values.

6.3 Part B – With Imbalance Handling

6.3.1 Class Weighting

Table 2: Model Comparison (Class Weighting)

Model	Accuracy	Balanced Acc.	ROC-AUC	F1 Macro	Recall (Disaster)	Recall (Non-Dis.)
Logistic Regression	0.8738	0.7866	0.8991	0.7895	0.6478	0.9254
Random Forest	0.8518	0.8014	0.8999	0.7753	0.7210	0.8817
Neural Network	0.8751	0.7737	0.8596	0.7850	0.6123	0.9352

Observation:

- Random Forest achieves the highest balanced accuracy (0.8014) and recall (0.7210), making it the most effective for detecting disaster tweets under class weighting.

- Neural Network attains the highest overall accuracy (0.8751) among the three, while maintaining competitive F1 Macro (0.7850).
- Logistic Regression offers a well-rounded performance with the highest ROC-AUC (0.8991) and stable scores across all metrics.
- Class weighting significantly improves minority class recall for all models compared to the baseline, particularly for Random Forest (+0.3026).

6.3.2 Random Oversampling (ROS)

Table 3: Model Comparison (Random Oversampling)

Model	Accuracy	Balanced Acc.	ROC-AUC	F1 Macro	Recall (Disaster)	Recall (Non-Dis.)
Logistic Regression	0.8751	0.7847	0.8944	0.7899	0.6407	0.9287
Random Forest	0.8562	0.8068	0.9040	0.7813	0.7281	0.8855
Neural Network	0.8681	0.7758	0.8659	0.7793	0.6288	0.9227

Observation:

- Random Forest achieves the best balanced accuracy (0.8068) and ROC-AUC (0.9040) among all models and all configurations, along with the highest recall (0.7281).
- Logistic Regression provides stable results with the highest accuracy (0.8751) and F1 Macro (0.7899) in this configuration.
- Neural Network shows moderate improvement over the baseline but still lags behind Logistic Regression and Random Forest in recall and ROC-AUC.
- Random oversampling yields the best overall minority class detection for Random Forest, outperforming even class weighting in both recall and ROC-AUC.

6.3.3 SMOTE (Synthetic Minority Oversampling Technique)

Table 4: Model Comparison (SMOTE)

Model	Accuracy	Balanced Acc.	ROC-AUC	F1 Macro	Recall (Disaster)	Recall (Non-Dis.)
Logistic Regression	0.8821	0.7762	0.8965	0.7931	0.6076	0.9449
Random Forest	0.8703	0.7890	0.9032	0.7872	0.6596	0.9184
Neural Network	0.8799	0.7749	0.8699	0.7903	0.6076	0.9422

Observation:

- Logistic Regression achieves the highest accuracy (0.8821) and F1 Macro (0.7931) across all configurations, making SMOTE its best-performing setup.

- Neural Network attains a competitive accuracy (0.8799) and F1 Macro (0.7903) under SMOTE, showing notable improvement over the baseline.
- Random Forest maintains the strongest ROC-AUC (0.9032) and highest recall (0.6596) within this configuration.
- SMOTE provides consistent and balanced improvements across all models, offering the best trade-off between accuracy and minority class detection for Logistic Regression and Neural Network.

7 Visualization of Results

This section provides a visual comparison of the model performances for the baseline scenario (Part A). These plots illustrate the trade-offs between precision and recall, as well as the ability of the models to distinguish between classes via ROC curves.

7.1 Overall Model Comparison



Figure 2: Full Model Comparison Across All Metrics

Observation:

- Logistic Regression with SMOTE achieves the highest accuracy and F1-score.
- Random Forest with ROS shows the best ROC-AUC and recall for disaster detection.
- Neural Network performs consistently but does not outperform LR or RF.
- Imbalance handling techniques significantly improve minority class metrics.

7.2 Model Performance Heatmap

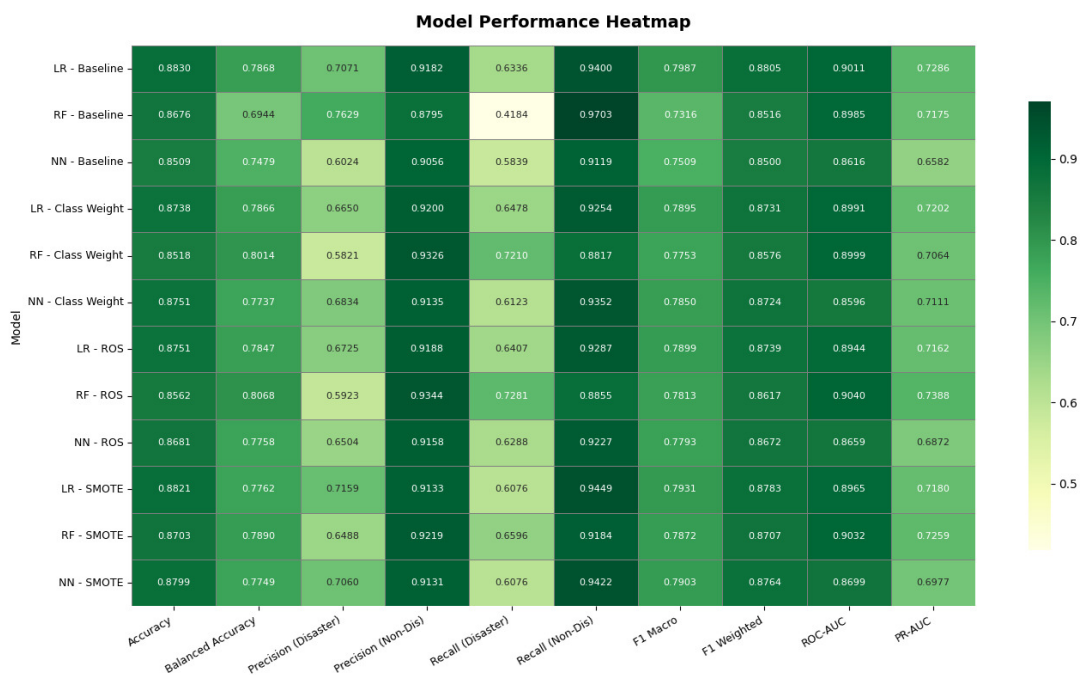


Figure 3: Model Performance Heatmap Across All Metrics

Observation:

- Darker regions indicate better performance across metrics.
- Logistic Regression with SMOTE shows strong performance across most metrics.
- Random Forest with ROS and SMOTE shows high recall and ROC-AUC values.
- Baseline models show weaker performance in recall for the disaster class.
- Heatmap clearly highlights the improvement after applying imbalance handling techniques.

7.3 Summary of Visualization Observations

- **Class Separation:** The ROC curves for Logistic Regression and Random Forest show high AUC values (≈ 0.90), indicating strong potential for class separation despite the imbalance.
- **Precision-Recall Gap:** The PR curves highlight the challenge of the minority class; while accuracy is high, the area under the PR curve is significantly lower than the ROC-AUC.
- **Confusion Matrix Bias:** The baseline confusion matrices show a high rate of False Negatives, reflecting the 18.59% minority class distribution.

8 Analysis and Discussion

This section presents the interpretation and comparison of model performance based on the overall comparison plots and heatmap. The results clearly show that class imbalance significantly affects model behavior, particularly leading to low recall for the minority (Disaster) class in baseline models, even when accuracy appears high.

From the visualizations, Logistic Regression consistently demonstrates strong and stable performance across most metrics, especially when combined with SMOTE, achieving the highest accuracy and F1-scores. Random Forest shows superior performance in terms of ROC-AUC and recall for the disaster class, particularly with Random Oversampling (ROS), making it more effective for detecting minority class instances. Neural Network models exhibit moderate performance and improvement after applying imbalance handling, but they do not outperform Logistic Regression or Random Forest.

The heatmap further highlights that imbalance handling techniques significantly improve minority class detection, with noticeable gains in recall and balanced accuracy across all models. Among the techniques, SMOTE provides the most balanced overall improvement, while ROS is particularly effective for enhancing recall in Random Forest. However, these improvements sometimes come with a slight trade-off in precision or overall accuracy.

Overall, the visual comparison confirms that applying imbalance handling techniques is essential for achieving reliable and balanced model performance, especially in scenarios with highly skewed class distributions.

Among the models, Random Forest performed best across most metrics, particularly with SMOTE. Logistic Regression remained stable and reliable, while the Neural Network showed variability and requires further tuning. Overall, SMOTE proved to be the most effective technique for improving generalization and achieving balanced performance.

9 Conclusion

- The project successfully implemented all required models and imbalance handling techniques.
- End-to-end execution produced cleaned data, model evaluation metrics, and visualization artifacts.
- The results highlight the expected trade-offs between minority class recall and precision.

9.1 Practical Recommendations

- If the priority is maximum disaster detection, Logistic Regression with Random Undersampling is recommended.
- If the priority is balanced overall performance, Random Forest with Class Weight is the best choice.
- If the priority is strong precision-recall performance for the minority class, Random Forest with Random Oversampling is preferred.